

Fix Direction: What the PR Should Actually Do

The Reasoning Chain

Step 1: Understand what Tile Engine actually is

TE is a two-part system:

Part A: Codegen

JSON config → unified_gemm_codegen.py → gemm_*.hpp → compiled binary
binary name: benchmark_gemm_universal_fp16_rcr_compv3_...

Part B: Runtime

benchmark_gemm_universal_fp16_rcr_compv3_... -m=1024 -n=1024 -k=1024
→ runs kernel, reports TFLOP/s

CI and downstream consumers depend on Part B — they invoke those benchmark binaries with problem sizes and read back performance numbers.

Step 2: Understand what the dispatcher already is

The dispatcher is also a two-part system:

Part A: Codegen (same Python script, different owner)

CMake → unified_gemm_codegen.py → gemm_*.hpp → compiled into libdispatcher_gemm_lib.so
This script lives in dispatcher/codegen/. It is the dispatcher's tool.

Part B: Runtime (Python interface, already built)

```
from ctypes_utils import DispatcherLib
lib = DispatcherLib.load()          # loads the .so
lib.run_gemm(A, B, C, M, N, K)      # dispatches to right kernel via registry
```

Critical observation: unified_gemm_codegen.py appears in both systems — but it belongs to the dispatcher. TE was using it too, but it was always the dispatcher's codegen. The ownership was always there.

Step 3: See what the current PR actually built

The PR built this:

```
TE JSON → te_to_dispatcher.py → config dict
                                     ↓
                                     drive_codegen.py (subprocess call to unified_gemm_codegen.p
y)
                                     ↓
                                     gemm_*.hpp
                                     ↓
                                     harness.cpp (custom single-kernel C++ runner)
                                     ↓
                                     compare output vs TE benchmark binary
```

There are three problems with this:

Problem A — The translator makes the dispatcher speak TE's language.

te_to_dispatcher.py takes TE JSON as input. That means the dispatcher is still downstream of TE's vocabulary. Vidya said: do not make the dispatcher look like Tile Engine. A translator that converts TE → dispatcher does exactly that — it keeps TE's JSON as the source of truth and makes the dispatcher adapt to it.

Problem B — drive_codegen.py treats the dispatcher's own codegen as a TE tool.

drive_codegen.py invokes unified_gemm_codegen.py as a subprocess from outside the dispatcher's build system. But that script already gets called by CMake when you build the dispatcher. The PR is re-invoking it manually, outside of CMake, to produce a one-off header for a custom harness. This bypasses the actual dispatcher build and produces a parallel, ad-hoc code path.

Problem C — The harness bypasses the dispatcher's Python interface.

The dispatcher already has DispatcherLib in ctypes_utils.py — a full Python runtime that loads the built .so and dispatches kernels. The PR ignored this and built harness.cpp from scratch. So the "parity" it proves is: "my custom single-kernel C++ runner agrees with TE." It never tests the actual dispatcher interface that would replace TE in production.

Step 4: What parity should actually mean

The Confluence page says: replace TE with a Python/JIT interface that drives the dispatcher. The thin frontend is the replacement.

So the parity that matters is:

TE benchmark binary	vs	DispatcherLib.run_gemm()
-m=1024 -n=1024 -k=1024		M=1024, N=1024, K=1024
reports 84.7 TFLOP/s		reports 84.7 TFLOP/s (± 2%)
output C matches CPU ref		output C matches CPU ref

That is the proof that DispatcherLib is a safe drop-in replacement for the TE benchmark binary. Once that is proven, you delete the TE binary and point whatever was calling it at DispatcherLib instead.

The current PR proves a different thing: "my translator + my custom harness agree with TE." That proof does not de-risk replacing TE with the dispatcher's actual Python interface, because the custom harness is not the dispatcher's Python interface.

Step 5: What the right approach looks like

1. Find the TE call sites
Where in CI, downstream scripts, or user-facing tools does something call benchmark_gemm_universal_* or invoke TE's JSON codegen?
These are the things that need to be replaced.
2. Confirm the dispatcher's .so already contains those kernels
Build the dispatcher normally (cmake + make).
Call DispatcherLib.load() and list all registered kernel identifiers.
Check that fp16/rcr/compv3/intrawave/etc. are all present.
They almost certainly are — the dispatcher already covers the full TE matrix.
3. Write a thin Python script using DispatcherLib that accepts the same inputs
TE callers say: "run fp16 rcr compv3 on M=1024, N=1024, K=1024"
The replacement says:
lib = DispatcherLib.load()
lib.run_gemm(A, B, C, M=1024, N=1024, K=1024, dtype="fp16", ...)
This is the "thin frontend" — TE conforms to the dispatcher model.

- 4. Prove parity through that interface
Run the same problem sizes through DispatcherLib and TE benchmark.
Assert numerical agreement and $\leq 2\%$ TFLOP/s delta.
No translator. No custom harness. No subprocess codegen.
- 5. Delete TE
Once parity is confirmed, remove the TE JSON $\rightarrow$ codegen $\rightarrow$ binary path.
The dispatcher's CMake build + DispatcherLib is the only system left.

Summary Table: Current PR vs Right Direction

Aspect	Current PR	Right Direction
Input	TE JSON config	TE call sites (CI scripts, benchmark invocations)
Translator	te_to_dispatcher.py converts TE vocab $\rightarrow$ dispatcher	No translator — dispatcher already has its own vocab
Codegen	drive_codegen.py invokes codegen as subprocess	CMake builds dispatcher normally; codegen is internal
Runtime tested	Custom harness.cpp single-kernel runner	DispatcherLib.run_gemm() — the actual replacement
Parity proven	Translator + custom harness agree with TE	DispatcherLib agrees with TE benchmark binaries
End state	Two systems still running, parity harness added	TE binaries deleted, DispatcherLib is the only path
Architectural direction	Dispatcher speaks TE's language	TE conforms to dispatcher model

The One-Sentence Summary

The current PR proved that a translation layer works. The project needs to prove that the dispatcher's own existing Python interface (DispatcherLib) is a drop-in replacement for TE benchmark binaries — and then delete those binaries. Those are different proofs, and only the second one enables deletion.